

CSCI 565: Assignment 2

This is an individual assignment. Do your own work. Do not copy / paste anything from any outside source. You may brainstorm with others on the high-level approaches if needed. It is intended that you figure this all out on your own, but if you get stuck, and have put in a full effort to resolve your issues on your own, you may come to office hours to get help if needed. My help will be brief and high level, giving a hint to point you in the right direction.

Objectives

- Starting with your assignment 1 work, create a multi-page HTML web site served from your local file system, with a navigation bar and separate pages for: The home page (the chess board), a History page, a Rules page, an About page.
- Generate the chessboard from JSON data accessed after page load via AJAX, in both vanilla Javascript and JQuery.
- Add a form under the chessboard and implement piece movement using DOM manipulation, both in vanilla Javascript and JQuery.

Steps

- Create a new folder csci565/assignments/asg2/ on your local file system, in the same folder you had previously created the asg1 folder.
- Create two sub-folders within asg2: v1_without_jquery and v2_with_jquery.
- Copy your asg1.html and asg1.css into the asg2/v1_without_jquery/ folder, and rename them chess.html and chess.css. Create a new file named chess.js in that same directory (empty for now).
- Add the appropriate **link** and **script** tags to the end of the **head** section of your **chess.html** page to include your **chess.css** and **chess.js** files (see the published Sudoku example that was included in this Canvas assignment as a guide).
- Split the **Rules**, **History** and **About** content from your newly copied **chess.html** file into new files names **rules.html**, **history.html** and **about.html**.
- Add the 4.4.1 bootstrap includes into the **head** section of each of your html files (see provided Sudoku example as a guide).
- Add a navigation bar to the top of your body section in each html file, with navigation elements pointing to your Home, Rules, History and About pages. Label the first navigation element Chess (see provided Sudoku example as a guide).
- Load v1_without_jquery/chess.html into your Chrome browser and test that your pages and navigation are rendering are working properly.
- Generate the board using DOM manipulation, from JSON data accessed via AJAX, both in vanilla javascript and in JQuery syntax. Replace your existing chessboard HTML with the following placeholder tags:

```
<div id="chessboard"></div>
```

You will use DOM manipulation to dynamically build the chessboard HTML tags as the inner html content of the above given html tags.

Create a file named chessboard.json within your v1_without_jquery directory, with the following contents:

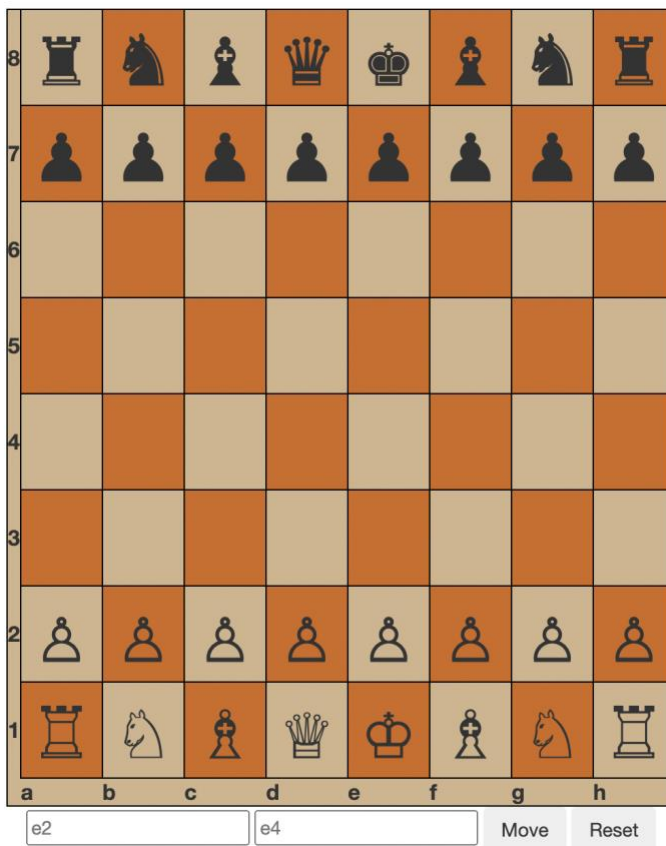
```
{
  "chessboard": [
    {"a8": "#9820;", "b8": "#9822;", "c8": "#9821;", "d8": "#9819;", "e8": "#9818;", "f8": "#9821;", "g8": "#9822;", "h8": "#9820;"},
    {"a7": "#9823;", "b7": "#9823;", "c7": "#9823;", "d7": "#9823;", "e7": "#9823;", "f7": "#9823;", "g7": "#9823;", "h7": "#9823;"},
    {"a6": " ", "b6": " ", "c6": " ", "d6": " ", "e6": " ", "f6": " ", "g6": " ", "h6": " "},
    {"a5": " ", "b5": " ", "c5": " ", "d5": " ", "e5": " ", "f5": " ", "g5": " ", "h5": " "},
    {"a4": " ", "b4": " ", "c4": " ", "d4": " ", "e4": " ", "f4": " ", "g4": " ", "h4": " "},
    {"a3": " ", "b3": " ", "c3": " ", "d3": " ", "e3": " ", "f3": " ", "g3": " ", "h3": " "},
    {"a2": "#9817;", "b2": "#9817;", "c2": "#9817;", "d2": "#9817;", "e2": "#9817;", "f2": "#9817;", "g2": "#9817;", "h2": "#9817;"}
  ]
}
```

```

{"a1": "&#9814;", "b1": "&#9816;", "c1": "&#9815;", "d1": "&#9813;", "e1": "&#9812;", "f1": "&#9815;", "g1": "&#9816;", "h1": "&#9814;"}
}

```

- Note: The ' ' characters in the JSON string above are the HTML entity for “non-breaking space”, a place-holder for rendering so that the browser doesn’t collapse any of the cells.
- Create an empty **render_chessboard()** javascript function in chessboard.js.
- Add an **onload** attribute to the <body> tag in chess.html, pointing it to your newly created render_chessboard() javascript function you created, like this:
<body onload="render_chessboard()" >
- Implement the code in the render_chessboard() to:
 - Download the json data from chessboard.json using AJAX, then create a JSON object from the data and use it to render the chessboard HTML table dynamically (see the provided example here as a guide):
https://www.ecst.csuchico.edu/~bherring/cins465/lectures/json/ajax_json_example_1.html).
- Load v1_without_jquery/chess.html into your Chrome browser and test that your pages, navigation and dynamic chessboard rendering are working properly.
- After the **v1_without_jquery** chessboard is rendering correctly, it's time to add the chess piece movement feature. Use the provided Sudoku example as a guide for adding a form to the bottom of the chessboard (a static HTML form is fine, you don’t need to generate it). Add the form just after the chessboard div tags.
- The board and form should render similar to the following image:



- By convention, chess moves are specified using the notation as shown above. For the starting position shown above, the move **e2 e4** means to advance the white pawn at square **e2** to square **e4**.
- Note: You do not need to determine if the given move is legal, just perform the moves designated by the user if they specified the source and destination locations in the correct format.

- Using a "**placeholder**" attribute on each **input** control, provide "hint" text as shown above ("e2" and "e4") for the expected input format.
- Add **id** attributes to uniquely identify each of the **input** controls. Using **id="src"** for the first input control and **id="dst"** for the bottom input control will likely be convenient.
- Create two javascript functions within chessboard.js, one for moving the chess pieces attached to the **Move** button (via the **onclick** attribute), and one for resetting the board to the starting state attached to the **Reset** button (also via the **onclick** attribute).
- Implement the javascript function for **Reset**. Hint: Use **window.location.reload()** to reload the page, which will reset all the pieces to their starting position.
- Add Bootstrap styled **input** and **button** controls similar to what is shown the image above. They don't have to match the provided image exactly, just make sure the controls are rendered below the chessboard, are styled with bootstrap, and are usable and functional. Hint: Use the **onClick** HTML attribute to handle the **Move** and **Reset** actions. See the provided Sudoku example as a guide.
- Implement the javascript function connected to the **Move** button. Use **document.getElementById(...).value** and **document.getElementById(...).innerHTML** to move the designated source piece to the designated destination square. Be sure to clear the source square after moving the piece. See the provided Sudoku example as a guide.
- Verify that the **Move** and **Reset** buttons are working as expected using the vanilla Javascript syntax described above. Enter a valid chess move (e.g. "e2" "e4") and select the **Move** button. Confirm that the chess piece moves correctly. Then select the Reset button and confirm that the chessboard resets to its starting position.
- Test the now complete version of your **v1_without_jquery** chessboard, including rendering, **Move** and **Reset**.
- Once everything is working, now would be a good time to back-up your work in progress (you may simply create a zip file for this and store it on a thumb drive, or push to a private github repo if you'd prefer).
- Now it's time to repeat the above steps for the **v2_with_jquery** version of your site. Start by copying all the files within your **v1_without_jquery** over to **v2_without_jquery**.
- Now systematically replace the vanilla javascript code you wrote with JQuery syntax instead. (see the provided example here as a guide):
https://www.ecst.csuchico.edu/~bherring/cins465/lectures/json/ajax_json_example_3.html
- Test everything again, zip the contents of your asg2 folder creating file asg2.zip, and submit to Canvas before the due date.